

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №0
по курсу «Алгоритмы и структуры данных»
Тема: Введение

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлунин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту.....	3
Задача №1. Ввод-вывод.....	3
Задача №2. Число Фибоначчи.....	9
Задача №3. Еще про числа Фибоначчи.....	15
Задача №4. Тестирование ваших алгоритмов.....	20
Вывод.....	24

Задачи по варианту

Задача №1. Ввод-вывод

Вам необходимо выполнить 4 следующих задачи:

1. Задача $a + b$. В данной задаче требуется вычислить сумму двух заданных чисел. Вход: одна строка, которая содержит два целых числа a и b . Для этих чисел выполняются условия $-10^9 \leq a, b \leq 10^9$. Выход: единственное целое число – результат сложения $a + b$.

2. Задача $a + b^2$. В данной задаче требуется вычислить значение $a + b^2$. Вход: одна строка, которая содержит два целых числа a и b . Для этих чисел выполняются условия $-10^9 \leq a, b \leq 10^9$. Выход: единственное целое число – результат сложения $a + b^2$.

3. Выполните задачу $a + b$ с использованием файлов.

- Имя входного файла: input.txt

- Имя выходного файла: output.txt

- Формат входного файла. Входной файл состоит из одной строки, которая содержит два целых числа a и b . Для этих чисел выполняются условия $-10^9 \leq a, b \leq 10^9$.

- Формат выходного файла. Выходной файл единственное целое число – результат сложения $a + b$.

Примеры

input.txt	12 25	130 61
output.txt	37	191

4. Выполните задачу $a + b^2$ с использованием файлов аналогично предыдущему пункту.

Листинг кода.

```
def suma():
    a, b = map(int, input().split())
    return a + b

def sum_square():
    a, b = map(int, input().split())
    return a + b**2
```

```
def sum_file():
    with open('input.txt') as f:
        a, b = map(int, f.readline().split())

    with open('output.txt', 'w') as f:
        f.write(str(a + b))

    return a + b

def sum_square_file():
    with open('input.txt') as f:
        a, b = map(int, f.readline().split())

    with open('output.txt', 'w') as f:
        f.write(str(a + b**2))

    return a + b**2
```

```
def summa():
    a, b = map(int, input().split())
    return a + b
```

```
def sum_square():
    a, b = map(int, input().split())
    return a + b**2
```

```
def sum_file():
    with open('input.txt') as f:
        a, b = map(int, f.readline().split())
    with open('output.txt', 'w') as f:
        f.write(str(a + b))
    return a + b
```

```
def sum_square_file():
    with open('input.txt') as f:
        a, b = map(int, f.readline().split())
    with open('output.txt', 'w') as f:
        f.write(str(a + b**2))
    return a + b**2
```

Текстовое объяснение решения.

1. Для выполнения первого пункта задания была написана функция `summa()`, которая не принимает аргументов, но считывает данные из строки ввода по заданному формату. Функция возвращает сумму введенных чисел.

Результат работы кода:

```
/Users/elizabethkulakova/.virtualenvs/  
12 25  
37  
  
Process finished with exit code 0
```

Рисунок 1 – Результат работы функции `summa()` на примере 1 из текста

```
/Users/elizabethkulakova/.virtualenvs/  
130 61  
191  
  
Process finished with exit code 0
```

Рисунок 2 – Результат работы функции `summa()` на примере 2 из текста

2. Для реализации второго пункта была написана функция `sum_square()`, которая не принимает аргументов, но считывает данные из строки ввода по заданному формату. Функция возвращает сумму первого числа и второго числа, возведенного в квадрат.

```
/Users/elizabethkulakova/.virtualenvs/  
12 25  
637  
  
Process finished with exit code 0
```

Рисунок 3 – Результат работы функции `sum_square()` на примере 1 из текста

```

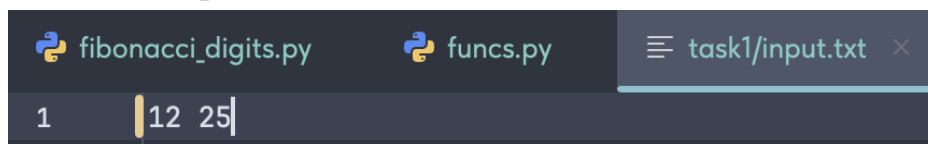
/Users/elizabethkulakova/.virtualenvs/
130 61
3851

Process finished with exit code 0

```

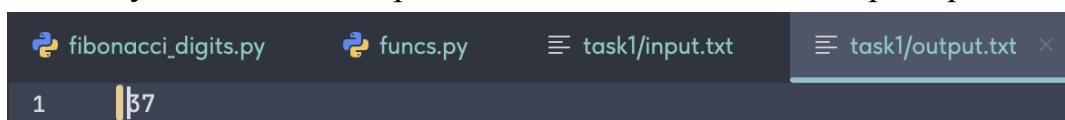
Рисунок 4 – Результат работы функции `sum_square()` на примере 2 из текста

3. Для выполнения третьего пункта задания была написана функция `sum_file()`, которая не принимает аргументов, но считывает данные из файла `input.txt`, где они записаны в определенном заданием формате. Функция вычисляет сумму считанных чисел, открывает файл `output.txt` на запись и фиксирует результат там, возвращает результат арифметической операции.



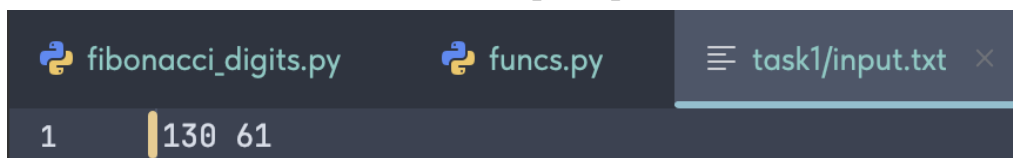
The screenshot shows a code editor with three tabs: `fibonacci_digits.py`, `funcs.py`, and `task1/input.txt`. The `task1/input.txt` tab is active, showing the text `1 12 25` on a single line.

Рисунок 5 – Файл `input.txt` с вводными данными примера 1



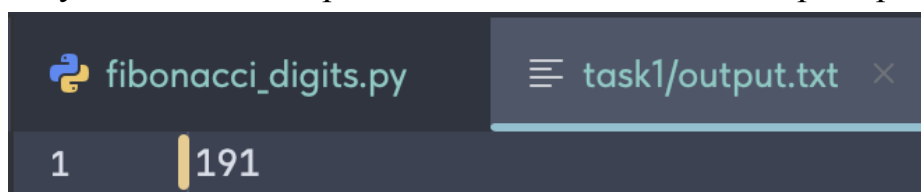
The screenshot shows a code editor with four tabs: `fibonacci_digits.py`, `funcs.py`, `task1/input.txt`, and `task1/output.txt`. The `task1/output.txt` tab is active, showing the text `1 37` on a single line.

Рисунок 6 – Файл `output.txt` после выполнения функции `sum_file()` с данными примера 1



The screenshot shows a code editor with three tabs: `fibonacci_digits.py`, `funcs.py`, and `task1/input.txt`. The `task1/input.txt` tab is active, showing the text `1 130 61` on a single line.

Рисунок 7 – Файл `input.txt` с вводными данными примера 2

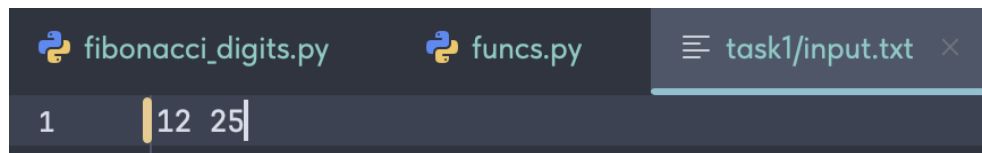


The screenshot shows a code editor with two tabs: `fibonacci_digits.py` and `task1/output.txt`. The `task1/output.txt` tab is active, showing the text `1 191` on a single line.

Рисунок 8 – Файл `output.txt` после выполнения функции `sum_file()` с данными примера 2

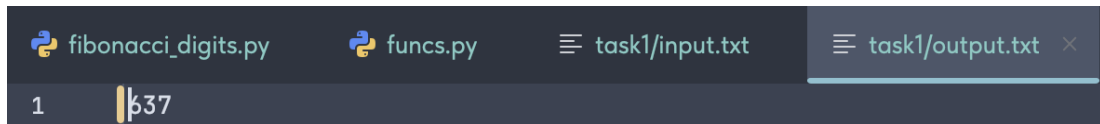
4. Для выполнения четвертого пункта задания была написана функция `sum_square_file()`, которая не принимает аргументов, но считывает данные из файла `input.txt`, где они записаны в определенном заданием формате. Функция вычисляет сумму первого числа и второго

числа, возведенного в квадрат, открывает файл output.txt на запись и фиксирует результат там, возвращает результат операции.



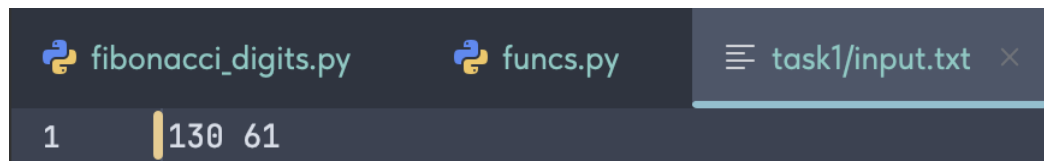
```
fibonacci_digits.py  funcs.py  task1/input.txt x
1  | 12 25|
```

Рисунок 9 – Файл input.txt с вводными данными примера 1



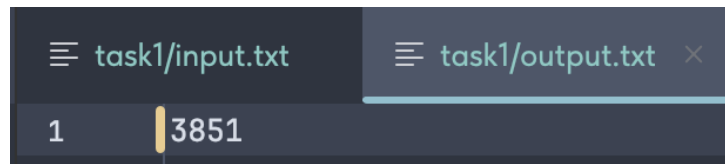
```
fibonacci_digits.py  funcs.py  task1/input.txt  task1/output.txt x
1  | 37
```

Рисунок 10 – Файл output.txt после выполнения функции sum_square_file() с данными примера 1



```
fibonacci_digits.py  funcs.py  task1/input.txt x
1  | 130 61
```

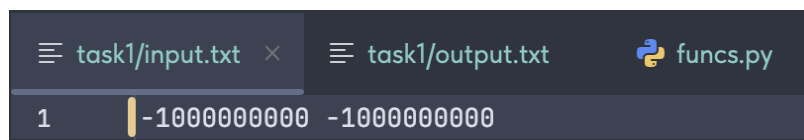
Рисунок 11 – Файл input.txt с вводными данными примера 2



```
task1/input.txt  task1/output.txt x
1  | 3851
```

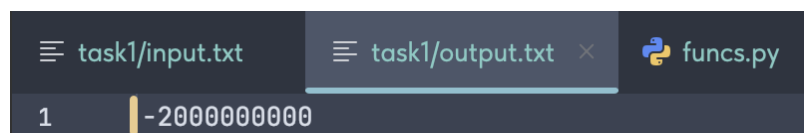
Рисунок 12 – Файл output.txt после выполнения функции sum_square_file() с данными примера 2

Результат работы кода на максимальных и минимальных значениях:



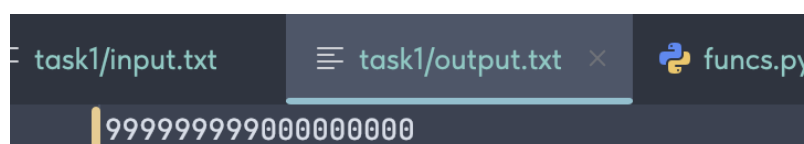
```
task1/input.txt x  task1/output.txt  funcs.py
1  | -10000000000 -10000000000
```

Рисунок 13 – Файл input.txt с вводными данными – минимальными значениями



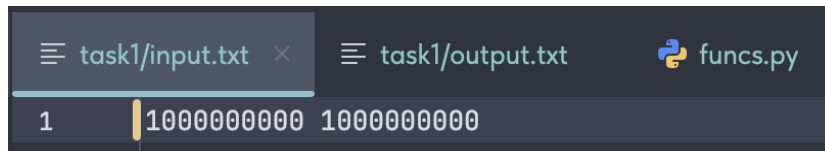
```
task1/input.txt  task1/output.txt x  funcs.py
1  | -20000000000
```

Рисунок 14 – Файл output.txt после выполнения функции sum_file() с минимальными значениями



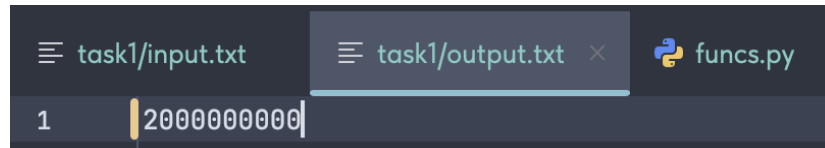
```
task1/input.txt  task1/output.txt x  funcs.py
1  | 9999999990000000000
```

Рисунок 15 – Файл output.txt после выполнения функции sum_square_file() с минимальными значениями



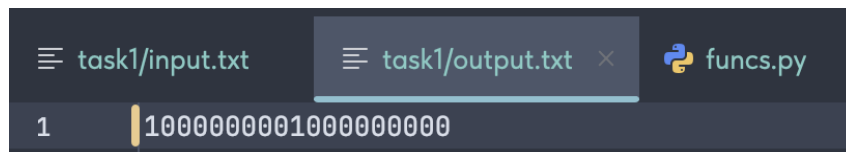
```
task1/input.txt × task1/output.txt funcs.py
1 10000000000 10000000000
```

Рисунок 16 – Файл input.txt с вводными данными – максимальными значениями



```
task1/input.txt task1/output.txt × funcs.py
1 20000000000
```

Рисунок 17 – Файл output.txt после выполнения функции sum_file() с максисмальными значениями



```
task1/input.txt task1/output.txt × funcs.py
1 1000000000010000000000
```

Рисунок 18 – Файл output.txt после выполнения функции sum_square_file() с максимальными значениями

Вывод по задаче: В этой задаче были рассмотрены базовые арифметические операции, а также работа с файлами (считывание информации из файла и запись результат выполнения операции в файл).

Задача №2. Число Фибоначчи

Определение последовательности Фибоначчи:

$$F_0 = 0 \ (1)$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \text{ для } i \geq 2.$$

Таким образом, каждое число Фибоначчи представляет собой сумму двух предыдущих, что дает последовательность 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Ваша цель – разработать эффективный алгоритм для подсчета чисел Фибоначчи. Вам предлагается начальный код на Python, который содержит наивный рекурсивный алгоритм:

```
def calc_fib(n):  
    if (n <= 1):  
        return n  
    return calc_fib(n - 1) + calc_fib(n - 2)  
n = int(input())  
print(calc_fib(n))
```

- Имя входного файла: input.txt
- Имя выходного файла: output.txt
- Формат входного файла. Целое число n . $0 \leq n \leq 45$.
- Формат выходного файла. Число F_n .

Пример.

input.txt	10
output.txt	55

Листинг кода.

```
def effect_fibs():  
    with open('input.txt') as f:  
        n = int(f.readline())  
  
    if 0 <= n <= 20577:  
        n_bit_len = n.bit_length()  
        cur_pos = 0  
        cur_val, next_val = (0, 1)  
        for i in range(n_bit_len):  
            cur_pos *= 2  
            cur_val, next_val = (cur_val*(2*next_val -  
cur_val), cur_val**2 + next_val**2)
```

```

        if (n >> n_bit_len - 1 - i) & 1:
            cur_pos += 1
            cur_val, next_val = (next_val,
cur_val+next_val)

        with open('output.txt', 'w') as f:
            f.write(str(cur_val))

    return cur_val
return 'input value cannot be out of range 0 - 20577!'

```

```

def effect_fibs():
    with open('input.txt') as f:
        n = int(f.readline())

    if 0 <= n <= 20577:
        n_bit_len = n.bit_length()
        cur_pos = 0
        cur_val, next_val = (0, 1)
        for i in range(n_bit_len):
            cur_pos *= 2
            cur_val, next_val = (cur_val*(2*next_val - cur_val), cur_val**2 +
next_val**2)

            if (n >> n_bit_len - 1 - i) & 1:
                cur_pos += 1
                cur_val, next_val = (next_val, cur_val+next_val)

        with open('output.txt', 'w') as f:
            f.write(str(cur_val))

    return cur_val
return 'input value cannot be out of range 0 - 20577!'

```

Текстовое объяснение решения.

Рекурсивный алгоритм для вычисления числа Фибоначчи обладает вычислительной сложностью $O(2^n)$. Чтобы сократить ресурсо- и времязатраты, можно воспользоваться итеративным подходом, чтобы свести вычислительную сложность к $O(\log_2(n))$. Это можно сделать,

воспользовавшись двоичной записью вводимого числа (номера числа Фибоначчи). Зная формулу числа Фибоначчи $f(n) = f(n-2) + f(n-1)$, получим следующие рассуждения, последовательно раскладывая слагаемые суммы по формуле:

$$f(n) = 1f(n) = 0f(n-1) + 1f(n-0);$$

$$f(n) = 0f(n-1) + 1(f(n-1) + f(n-2)) = 1f(n-2) + 1f(n-1);$$

$$f(n) = 1f(n-2) + 1(f(n-2) + f(n-3)) = 1f(n-3) + 2f(n-2);$$

$$f(n) = 1f(n-3) + 2(f(n-3) + f(n-4)) = 2f(n-4) + 3f(n-3);$$

$$f(n) = 2f(n-4) + 3(f(n-4) + f(n-5)) = 3f(n-5) + 5f(n-4);$$

$$f(n) = 3f(n-5) + 5(f(n-5) + f(n-6)) = 5f(n-6) + 8f(n-5);$$

$$f(n) = 5f(n-6) + 8(f(n-6) + f(n-7)) = 8f(n-7) + 13f(n-6);$$

Заметим, что коэффициенты при слагаемых образуют ряд Фибоначчи.

Таким образом, мы можем получить формулу вида $f(n) = f(x)f(n-(x+1)) + f(x+1)f(n-x)$. Эта формула справедлива как для $x = x+1$, так и для $x = x-1$ (доказывается по индукции).

Положим $n=a+b$, $x=a$. Тогда $f(a+b) = f(a)f(a+b-(a+1)) + f(a+1)f(a+b-a) = f(a)f(b-1) + f(a+1)f(b) = f(a)f(b-1) + f(b)(f(a) + f(a-1))$. Пусть $a=b$, тогда получаем выражение для удвоенного числа: $f(2a) = f(a)f(a-1) + f(a)f(a) + f(a-1)f(a) = f(a)(2f(a-1) + f(a))$.

Так как нам для вычисления нужно значение предыдущего числа Фибоначчи, то рассмотрим $f(a-1+b) = f(a+b-1) = f(a-1)f(b-1) + f(b)(f(a-1) + f(a-2)) = f(a-1)f(b-1) + f(a)f(b)$. Аналогично для следующего: $f(a+1+b) = f(a+b+1) = f(a+1)f(b-1) + f(b)(f(a+1) + f(a)) = f(a+1)f(b+1) + f(a)f(b)$. Вновь положим, что $a=b$, и получим $f(2a-1) = f(a-1)^2 + f(a)^2$, $f(2a+1) = f(a+1)^2 + f(a)^2$.

Таким образом, мы получили необходимые для быстрого алгоритма формулы: $f(2n-1) = f(n-1)^2 + f(n)^2$, $f(2n) = f(n)(2f(n-1) + f(n))$ и $f(2n+1) = f(n+1)^2 + f(n)^2$.

Алгоритм быстрого вычисления строится на битовом представлении вводимого числа n . Чтобы не вычислять каждое следующее значение при итерации от 0 до n , как раз нужно использовать выведенные выше формулы $f(2n) = f(n)(2f(n-1) + f(n))$ (1) и $f(2n+1) = f(n+1)^2 + f(n)^2$ (2). Благодаря им есть возможность добраться до искомого значения соответствующего номера.

Функция `effect_fibs()` считывает из файла `input.txt` позицию, значение на которой мы хотим получить, а далее с помощью встроенной функции `bit_length()` вычисляется длина значения в битовом представлении:

```
def effect_fibs():
    with open('input_extra.txt') as f:
        n = int(f.readline())

    n_bit_len = n.bit_length()
```

Итак, сначала в коде мы устанавливаем уже известные нам значения для $f(0)$ и $f(1)$ в `cur_val` и `next_val` соответственно, а также обозначаем текущую позицию в переменной `cur_pos`:

```
cur_pos = 0
cur_val, next_val = (0, 1)
```

Далее начинается итерация, в процессе которой на каждом шаге позиция удваивается и вычисляются значения на новой удвоенной позиции и следующее за ним по выведенным формулам (1) и (2):

```
for i in range(n_bit_len):
    cur_pos *= 2
    cur_val, next_val = (cur_val*(2*next_val - cur_val), cur_val**2 +
next_val**2)
```

Далее оцениваем значение бита, на номере которого находимся в итерации, начиная со старшего бита и заканчивая младшим. Если бит 1, то увеличиваем позицию на 1 и значения соответственно:

```
if (n >> n_bit_len - 1 - i) & 1:
    cur_pos += 1
    cur_val, next_val = (next_val, cur_val+next_val)
```

Таким образом мы добираемся до числа на искомой позиции не за n шагов, а за $\log_2 n$ за счет представления позиции числа Фибоначчи в битовом представлении. Получаем результат, записываем его в файл `output.txt` и возвращаем:

```
with open('output_extra.txt', 'w') as f:
    f.write(str(cur_val))
```

```
return cur_val
```

Также с помощью условного оператора ловится значение вне заданного диапазона.

Результат работы кода на примерах из задания:

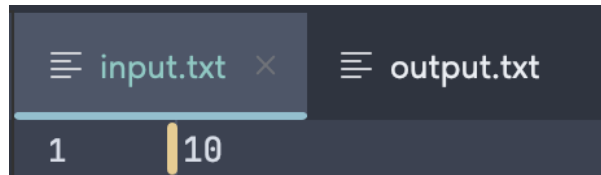


Рисунок 19 – Вводимое значение номера числа Фибоначчи в input.txt

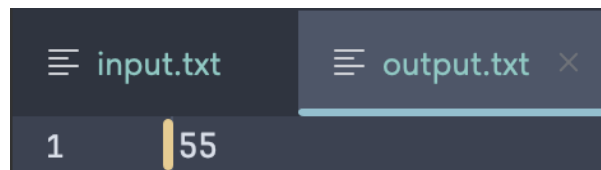


Рисунок 20 – Результат работы кода для значения 10, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

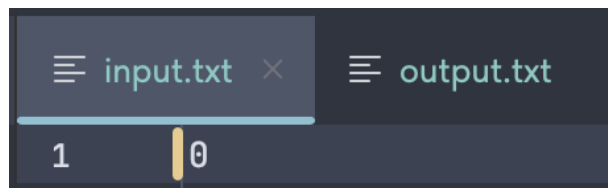


Рисунок 21 – Минимальное вводимое значение номера числа Фибоначчи в input.txt

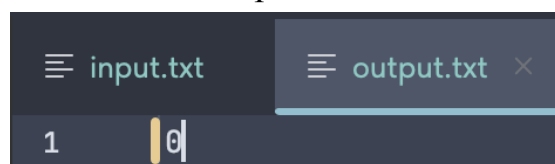


Рисунок 22 – Результат работы кода для значения 0, записанный в output.txt

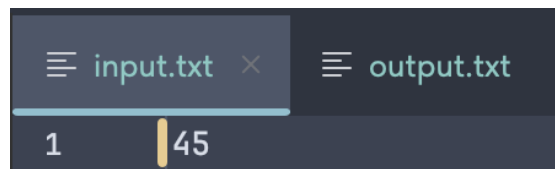


Рисунок 23 – Максимальное вводимое значение номера числа Фибоначчи в input.txt

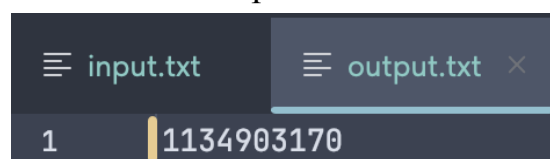


Рисунок 24 – Результат работы кода для значения 45, записанный в
output.txt

Вывод по задаче: Реализован эффективный алгоритм вычисления числа Фибоначчи по заданному номеру, изучена работа с битовым представлением числа и связанные операции.

Задача №3. Еще про числа Фибоначчи

Определение последней цифры большого числа Фибоначчи. Числа Фибоначчи растут экспоненциально. Например, $F_{200} = 280571172992510140037611932413038677189525$

Хранить такие суммы в массиве, и при этом подсчитывать сумму, будет достаточно долго. Найти последнюю цифру любого числа достаточно просто: $F \bmod 10$.

- Имя входного файла: input.txt
- Имя выходного файла: output.txt
- Формат входного файла. Целое число n . $0 \leq n \leq 10^7$
- Формат выходного файла. Одна последняя цифра числа F_n .

Пример 1

input.txt	331
output.txt	9

$F_{331} = 668996615388005031531000081241745415306766517246774551964595292186469$.

Пример 2

input.txt	327305
output.txt	5

Это число не влезет в страницу, но оканчивается действительно на 5.

- Ограничение по времени: 5сек.
- Ограничение по памяти: 512 мб.

Листинг кода.

```
def last_fib_digit():
    with open('input.txt') as f:
        n = int(f.readline())

    if n >= 0:
        equiv_fib = n % 60
        if equiv_fib <= 1:
            val = equiv_fib
        else:
            val = effect_fibs(equiv_fib) % 10
```

```

        with open('output.txt', 'w') as f:
            f.write(str(val))
        return val
    return 'input value cannot be negative!'

```

```

def last_fib_digit():
    with open('input.txt') as f:
        n = int(f.readline())

    if n >= 0:
        equiv_fib = n % 60
        if equiv_fib <= 1:
            val = equiv_fib
        else:
            val = effect_fibs(equiv_fib) % 10

        with open('output.txt', 'w') as f:
            f.write(str(val))
        return val
    return 'input value cannot be negative!'

```

Адаптированная для задачи функция effect_fibs():

```

def effect_fibs(position):
    if position >= 0:
        n_bit_len = position.bit_length()
        cur_pos = 0
        cur_val, next_val = (0, 1)
        for i in range(n_bit_len):
            cur_pos *= 2
            cur_val, next_val = (cur_val * (2 * next_val -
cur_val), cur_val ** 2 + next_val ** 2)

            if (position >> n_bit_len - 1 - i) & 1:
                cur_pos += 1
                cur_val, next_val = (next_val, cur_val +
next_val)

        return cur_val
    return 'input value cannot be negative!'

```

```

def effect_fibs(position):
    if position >= 0:

```

```

n_bit_len = position.bit_length()
cur_pos = 0
cur_val, next_val = (0, 1)
for i in range(n_bit_len):
    cur_pos *= 2
    cur_val, next_val = (cur_val * (2 * next_val - cur_val), cur_val ** 2 +
next_val ** 2)

    if (position >> n_bit_len - 1 - i) & 1:
        cur_pos += 1
        cur_val, next_val = (next_val, cur_val + next_val)

return cur_val
return 'input value cannot be negative!'

```

Текстовое объяснение решения.

Можно заметить, что последняя цифра числа Фибоначчи повторяется с периодом 60 (период Пизано для числа Фибоначчи по модулю 10). Поэтому алгоритм строится достаточно просто с задействованием функции `effect_fibs()` из предыдущего задания.

Функция `last_fib_digit()` читает данные из файла `input.txt` и фиксирует считанное значение в переменной `n`. Производится поиск остатка от деления `n` на 60 и записывается в переменную `equiv_fib`, далее функция проводит проверку на заранее известные значения 0 и 1. При прохождении условия итоговое значение сразу фиксируется в переменную `val` для экономии времени и затрат памяти. Если же известные значения использовать сразу не получится, то функция передает `effect_fibs()` (функция изменена под задачу, файлы для экономии времени и памяти не использует) значение переменной `equiv_fib`. Далее в переменную `val` записывается значение остатка от деления на 10 результата выполнения функции `effect_fibs()`. В конце переменная `val` записывается в файл `output.txt` и возвращается.

Также с помощью условного оператора ловится значение вне заданного диапазона.

Результат работы кода на примерах из задания:

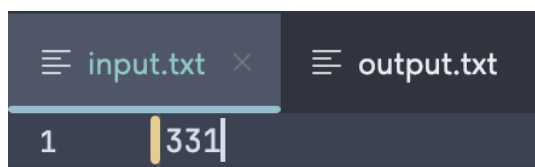


Рисунок 25 – Входные данные значения номера числа Фибоначчи в input.txt из примера 1

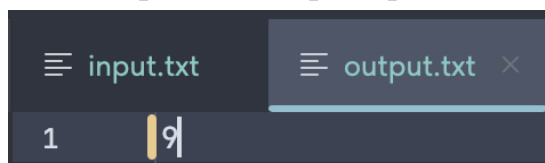


Рисунок 26 – Результат работы кода для входных данных из примера 1, записанный в output.txt

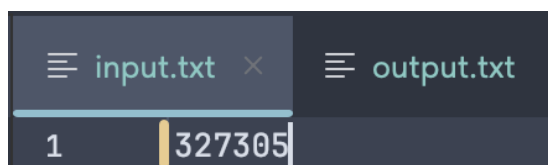


Рисунок 27 – Входные данные значения номера числа Фибоначчи в input.txt из примера 2

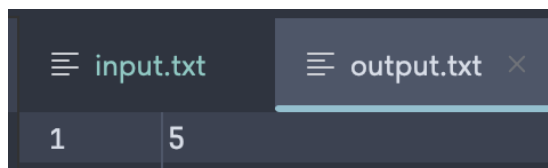


Рисунок 28 – Результат работы кода для входных данных из примера 2, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

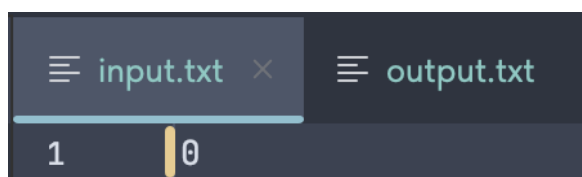


Рисунок 29 – Минимальное вводимое значение номера числа Фибоначчи в input.txt

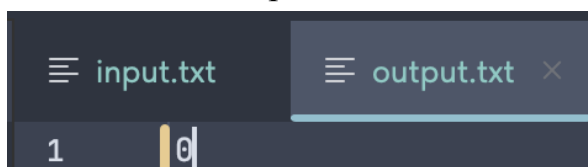


Рисунок 30 – Результат работы кода для значения 0, записанный в output.txt

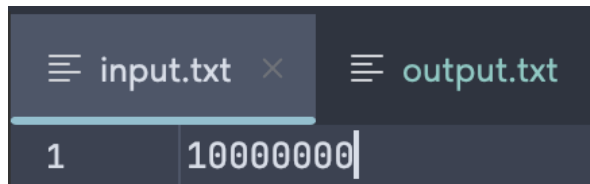


Рисунок 31 – Максимальное вводимое значение номера числа Фибоначчи в input.txt

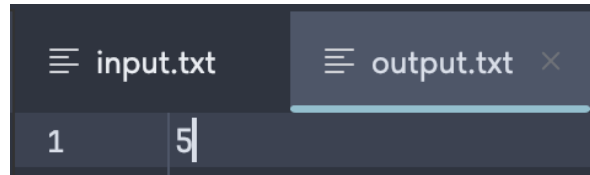


Рисунок 32 – Результат работы кода для значения 10^7 , записанный в output.txt

Вывод по задаче: С помощью функции из предыдущего задания был реализован алгоритм поиска последней цифры для любого числа Фибоначчи. Было выяснено, что период Пизано числа Фибоначчи по модулю 10 – 60.

Задача №4. Тестирование ваших алгоритмов.

Задача: вам необходимо протестировать время выполнения вашего алгоритма в Задании 2 и Задании 3.

Дополнительно: вы можете протестировать объем используемой памяти при выполнении вашего алгоритма.

Листинг кода.

```
import time
import tracemalloc
from functools import wraps

def measure_memory_usage(func):
    def wrapper(*args, **kwargs):
        tracemalloc.start()

        snapshot1 = tracemalloc.take_snapshot()
        # Call the original function
        result = func(*args, **kwargs)

        snapshot2 = tracemalloc.take_snapshot()
        stats = snapshot2.compare_to(snapshot1, "filename")

        # Print the top memory-consuming lines
        print(f"Memory usage of {func.__name__}:")
        for stat in stats:
            print(stat)

        # Return the result
        return result

    return wrapper

def timeit(func):
    @wraps(func)
    def timeit_wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        end_time = time.perf_counter()
        total_time = end_time - start_time
        with open('input.txt') as f:
            input_val = f.readline()
```

```

        print(f'Function {func.__name__}({input_val}) took
{total_time:.4f} seconds')
        return result

    return timeit_wrapper

```

```
import time
```

```
import tracemalloc
```

```
from functools import wraps
```

```
def measure_memory_usage(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        tracemalloc.start()
```

```
        snapshot1 = tracemalloc.take_snapshot()
```

```
        # Call the original function
```

```
        result = func(*args, **kwargs)
```

```
        snapshot2 = tracemalloc.take_snapshot()
```

```
        stats = snapshot2.compare_to(snapshot1, "filename")
```

```
        # Print the top memory-consuming lines
```

```
        print(f'Memory usage of {func.__name__}:')
```

```
        for stat in stats:
```

```
            print(stat)
```

```
        # Return the result
```

```
        return result
```

```
    return wrapper
```

```
def timeit(func):
```

```
    @wraps(func)
```

```
    def timeit_wrapper(*args, **kwargs):
```

```
        start_time = time.perf_counter()
```

```
        result = func(*args, **kwargs)
```

```
        end_time = time.perf_counter()
```

```
        total_time = end_time - start_time
```

```
        with open('input.txt') as f:
```

```

        input_val = f.readline()
        print(f'Function {func.__name__}({input_val}) took {total_time:.4f}
seconds')
        return result

    return timeit_wrapper

```

Текстовое объяснение решения.

Для выполнения задания было предложено использовать декораторы (функции, которые возвращают функции, внутри которых выполняется и исследуется целевая функция). Ими были обернуты функции `effect_fibs()` и `last_fib_digit()`.

Результат работы кода на примерах из заданий и минимальных и максимальных значениях:

Таблица для задачи 2

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0010 сек	243 байт
Пример из задачи 2	0.0008 сек	1382 байт
Верхняя граница диапазона значений входных данных из текста задачи	0.0005 сек	1449 байт

Таблица для задачи 3

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0017 сек	182 байт
Пример из задачи 3 (1)	0.0016 сек	1316 байт
Пример из задачи 3 (2)	0.0045 сек	1533 байт

Верхняя граница диапазона значений входных данных из текста задачи	0.0005 сек	1694 байт
---	------------	-----------

Вывод

В этой лабораторной удалось поработать с числами Фибоначчи, вывести и разработать эффективный алгоритм для их нахождения, а также с помощью эффективного вычисления разработать алгоритм для определения последней цифры любого числа Фибоначчи. Также удалось познакомиться с декораторами для анализа эффективности разработанных алгоритмов.